

Traffic Light Object Detection write up

1) We start by installing missing libraries required to execute the project which includes ultralytics and several other libraries. Please remember to restart kernel after installing libraries and packages.

2) Next we import required packages and libraries

3) Next we download and unzip the dataset using `download_and_unzip` function which takes dataset URL as input. This function uses `requests` and `zipfile` libraries to download and unzip the dataset.

4) Next we visualize few images from training dataset

5) Next we visualize few images from validation dataset with bounding box drawn on top of it

6) Next comes training part. Here we first create data yaml file that is used to guide training process. This includes custom class names, parent folder path along with relative train and validation dataset paths.

7) Next we create training configuration that specifies training image size, epochs for which model will be trained and batch size. We tried different training image size including 640, 736 and 768. We got the best results with 768 with 50-75mAP crossing several times with confidence. For other image size, we were getting there but it was hovering between 48-50 as max mAP. With 768 we crossed 50mAP right in 60th epoch itself.

Batch size of 18 was chosen to fit the GPU memory available (24GB) comfortably so that system doesn't run out of cuda memory.

8) Next we load YOLO 8 model. We chose `yolov8x(X)` version since we got the best results with it during experimentation.

9) Next we train YOLO model. Please note that training completed successfully, however, due to unknown jupyter related issues, system could not capture the logs beyond 100 epochs.. However, if you see we crossed the threshold of 48 mAP in 60th epoch itself. It does not make sense to repeat entire process again due to this issue. If you need I can provide the weight files. I have also validated it on validation dataset using trained model as well the reinitialized model and then loaded best weights. All the validation results are in sync. We got the best mAP(50-95) of 52.6 on validation dataset.

10) Next we choose 5 random images from validation dataset and run inference on it using `run_inference` function and then plot these images along with bounding boxes using `visualize` function

11) Next we download the video and run inference against it. We take the video created during inference process(in avi format) and convert to mp4 format.

12) Next we visualize the video by embedding it in jupyter notebook itself